



# Revision Notes: Bit Manipulation Techniques

## Overview

This class delved into different bit manipulation techniques that are essential for optimizing problem-solving skills, particularly in competitive programming and technical interviews. The session primarily focused on understanding and applying bitwise operators like AND, OR, XOR, and utilizing the left and right shift operators to solve problems.

## Key Concepts and Techniques

### Bitwise Operators

#### 1. Basic Operators: AND, OR, XOR

- **AND (&):** Used to clear bits and can identify even or odd numbers. If  $A \& 1$  gives  $0$ , the number is even; if  $1$ , the number is odd.
- **OR (|):** Used to set bits.
- **XOR ( $\oplus$ ):** A bitwise operation that returns  $1$  for every position in which the corresponding bits are different. It's instrumental in solving problems where elements occur in pairs except for one.

#### 2. Properties of XOR

- $A \oplus A = 0$ : This property is key in identifying unique elements where every other element occurs twice.
- $A \oplus 0 = A$ : XOR with zero returns the number itself.

## Bit Manipulation Techniques

#### 1. Setting a Specific Bit

- To ensure the  $i$ th bit of a number  $A$  is set, you can use the expression  $A | (1 \ll i)$ . This handles cases where the bit may already be set or unset.

#### 2. Checking a Specific Bit



### 3. Toggling a Bit

- Use XOR to toggle the state of a specific bit:  $A \oplus (1 \ll i)$ . This operation inverts the current state of the bit.

### 4. Clearing a Bit

- You can clear a specific bit using the expression  $A \& \sim(1 \ll i)$  where  $\sim$  denotes the NOT operation.

## Shift Operators

### 1. Left Shift Operator (<<)

- The left shift operator moves bits to the left, effectively multiplying the number by powers of two. For example,  $10 \ll 1$  results in  $20$ , and  $10 \ll 2$  results in  $40$ .
- It's vital to consider overflow conditions when dealing with large shifts beyond the bit capacity.

### 2. Right Shift Operator (>>)

- The right shift operator divides a number by powers of two, discarding the least significant bits. For example,  $10 \gg 1$  results in  $5$ .

## Problem-Solving with Bit Manipulation

### 1. Finding a Unique Element in an Array

- If every element occurs twice except one, XOR all numbers together. The result will be the number with a single occurrence because identical numbers cancel out.

### 2. Checking Even/Odd Number Using Bitwise Operators

- Use  $A \& 1$  to check if a number is even or odd. This is because the least significant bit (LSB) of even numbers is  $0$ , and for odd numbers, it is  $1$ .

### 3. Identifying Two Unique Numbers

- When two numbers in an array appear uniquely and all others appear twice, XOR helps to distinguish them. First XOR all numbers; the result will be the XOR of two unique numbers. By isolating any set bit in this result, you can split the array into two groups, each containing one unique number.



This class comprehensively covered the critical aspects of bit manipulation, with practical insights into how these operations can be applied efficiently to solve common algorithmic challenges. Mastery of these techniques is crucial for optimizing algorithms in competitive programming and various computing applications.